Database Source Code

Connection Config, Schema Builder & SQL Schema

Table of Contents

db/config.js

```
import "dotenv/config";
import path from "path";
import { fileURLToPath } from "url";
import mysql from "mysql2/promise";

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

// Database connection pool
export const db = mysql.createPool({
  host: process.env.DB_HOST || "localhost",
  user: process.env.DB_USER || "root",
  // Double-check your .env: usually this variable is DB_PASSWORD, not DB_PASS
  password: process.env.DB_PASS || "",
  database: process.env.DB_NAME || "evangadi_forum",
  waitForConnections: true,
  connectionLimit: 10,
  queueLimit: 0
});


const ensureParams = (params) => {
  if (params === undefined || params === null) {
    throw new Error("SQL parameters are required");
  }
  const isArray = Array.isArray(params);
  const isObject = !isArray && typeof params === "object";
  if (!isArray && !isObject) {
    throw new Error("SQL parameters must be an array or object");
  }
};

export const safeExecute = async (sql, params) => {
  if (typeof sql !== "string" || sql.trim().length === 0) {
    throw new Error("SQL query must be a non-empty string");
  }
  ensureParams(params);
  const [result] = await db.execute(sql, params);
  return result;
};

// import dotenv from "dotenv";
// dotenv.config();
// import mysql from "mysql2/promise";

// // Database connection pool
// export const db = mysql.createPool({
//   host: process.env.DB_HOST || "localhost",
//   port: process.env.DB_PORT || 8889,
//   user: process.env.DB_USER || "root",
//   password: process.env.DB_PASS || "",
//   database: process.env.DB_NAME || "forumPage",
// });

// const ensureParams = (params) => {
//   if (params === undefined || params === null) {
//     throw new Error("SQL parameters are required");
//   }
//   const isArray = Array.isArray(params);
//   const isObject = !isArray && typeof params === "object";
//   if (!isArray && !isObject) {
//     throw new Error("SQL parameters must be an array or object");
//   }
// };

// export const safeExecute = async (sql, params) => {
//   if (typeof sql !== "string" || sql.trim().length === 0) {
//     throw new Error("SQL query must be a non-empty string");
//   }
//   ensureParams(params);
//   const [result] = await db.execute(sql, params);
//   return result;
// };
```

db/schema.js

```
import { db } from "./config.js";

const createTables = async () => {
  try {
    await db.execute(`SET FOREIGN_KEY_CHECKS = 0`);

    // USERS
    await db.execute(`
      CREATE TABLE IF NOT EXISTS users (
        user_id INT AUTO_INCREMENT PRIMARY KEY,
        first_name VARCHAR(50) NOT NULL,
        last_name VARCHAR(50) NOT NULL,
        email VARCHAR(320) NOT NULL UNIQUE,
        password_hash VARCHAR(255) NOT NULL,
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
        CHECK (email = LOWER(email)),
        INDEX idx_users_email (email)
      ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
    `);

    // QUESTIONS
    await db.execute(`
      CREATE TABLE IF NOT EXISTS questions (
        question_id INT AUTO_INCREMENT PRIMARY KEY,
        question_hash CHAR(16) NOT NULL UNIQUE,
        user_id INT NOT NULL,
        title VARCHAR(255) NOT NULL,
        content TEXT NOT NULL,
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,

        CHECK (CHAR_LENGTH(title) >= 5),
        CHECK (CHAR_LENGTH(content) >= 10),

        FOREIGN KEY (user_id)
          REFERENCES users(user_id)
          ON DELETE CASCADE,

        INDEX idx_questions_user_id (user_id),
        INDEX idx_questions_created_at (created_at),
        FULLTEXT KEY ft_questions_search (title, content)
      ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
    `);

    // QUESTION VECTORS
    await db.execute(`
      CREATE TABLE IF NOT EXISTS question_vectors (
        vector_id BIGINT AUTO_INCREMENT PRIMARY KEY,
        question_id INT NOT NULL,
        source_text TEXT NOT NULL,
        embedding JSON NOT NULL,
        status VARCHAR(20) DEFAULT 'ready',
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,

        FOREIGN KEY (question_id)
          REFERENCES questions(question_id)
          ON DELETE CASCADE,

        UNIQUE KEY uniq_question_vectors_question_id (question_id)
      ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
    `);

    // ANSWERS
    await db.execute(`
      CREATE TABLE IF NOT EXISTS answers (
        answer_id INT AUTO_INCREMENT PRIMARY KEY,
        question_id INT NOT NULL,
        user_id INT NOT NULL,
        content TEXT NOT NULL,
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,

        FOREIGN KEY (question_id)
          REFERENCES questions(question_id)
          ON DELETE CASCADE,

        FOREIGN KEY (user_id)
          REFERENCES users(user_id)
          ON DELETE CASCADE,

        INDEX idx_answers_question_id (question_id),
        INDEX idx_answers_user_id (user_id),
        INDEX idx_answers_created_at (created_at)
      ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
    `);

    // DOCUMENTS
    await db.execute(`
      CREATE TABLE IF NOT EXISTS documents (
```

```
        document_id INT AUTO_INCREMENT PRIMARY KEY,
        user_id INT NOT NULL,
        title VARCHAR(512) NOT NULL,
        mime_type VARCHAR(128) NOT NULL DEFAULT 'application/pdf',
        storage_path VARCHAR(1024) NOT NULL,
        byte_size BIGINT NOT NULL DEFAULT 0,
        status VARCHAR(20) NOT NULL DEFAULT 'pending',
        error_message TEXT NULL,
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,

        FOREIGN KEY (user_id)
          REFERENCES users(user_id)
          ON DELETE CASCADE,

        INDEX idx_documents_user_created (user_id, created_at)
      ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
    `);

    // DOCUMENT CHUNKS
    await db.execute(`
      CREATE TABLE IF NOT EXISTS document_chunks (
        chunk_id BIGINT AUTO_INCREMENT PRIMARY KEY,
        document_id INT NOT NULL,
        chunk_index INT NOT NULL,
        content TEXT NOT NULL,
        page_start INT NULL,
        page_end INT NULL,
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

        FOREIGN KEY (document_id)
          REFERENCES documents(document_id)
          ON DELETE CASCADE,

        UNIQUE KEY uniq_document_chunks_doc_index (document_id, chunk_index),
        INDEX idx_document_chunks_document_id (document_id)
      ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
    `);

    // DOCUMENT CHUNK VECTORS
    await db.execute(`
      CREATE TABLE IF NOT EXISTS document_chunk_vectors (
        chunk_vector_id BIGINT AUTO_INCREMENT PRIMARY KEY,
        chunk_id BIGINT NOT NULL,
        source_text TEXT NOT NULL,
        embedding JSON NOT NULL,
        status VARCHAR(20) NOT NULL DEFAULT 'ready',
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,

        FOREIGN KEY (chunk_id)
          REFERENCES document_chunks(chunk_id)
          ON DELETE CASCADE,

        UNIQUE KEY uniq_chunk_vectors_chunk_id (chunk_id)
      ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
    `);

    await db.execute(`SET FOREIGN_KEY_CHECKS = 1`);

    console.log("✅ All Evangadi Forum tables created successfully.");
    process.exit(0);
  } catch (error) {
    console.error("❌ Error creating tables:", error);
    process.exit(1);
  }
};

createTables();
```

db/schema.sql

```sql
-- Database Schema for Evangadi Forum
-- Platform: MySQL

SET FOREIGN_KEY_CHECKS = 0;

-- --------------------------------------------------------------------------------
-- 1. Users Table
-- Stores user account information.
-- --------------------------------------------------------------------------------
DROP TABLE IF EXISTS `users`;
CREATE TABLE `users` (
    `user_id` INT AUTO_INCREMENT PRIMARY KEY,
    `first_name` VARCHAR(50) NOT NULL,
    `last_name` VARCHAR(50) NOT NULL,
    `email` VARCHAR(320) NOT NULL UNIQUE,
    `password_hash` VARCHAR(255) NOT NULL,
    `created_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    `updated_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    CHECK (`email` = LOWER(`email`)),

    INDEX `idx_users_email` (`email`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- --------------------------------------------------------------------------------
-- 2. Questions Table
-- Stores the main questions posted by users.
-- Supports full-text search on title and content for exact match search.
-- --------------------------------------------------------------------------------
DROP TABLE IF EXISTS `questions`;
CREATE TABLE `questions` (
    `question_id` INT AUTO_INCREMENT PRIMARY KEY,
    `question_hash` CHAR(16) NOT NULL UNIQUE, -- Used for /question/:hash routing
    `user_id` INT NOT NULL,
    `title` VARCHAR(255) NOT NULL,
    `content` TEXT NOT NULL, -- Detailed content including code sections
    `created_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    `updated_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    CHECK (CHAR_LENGTH(`title`) >= 5),
    CHECK (CHAR_LENGTH(`content`) >= 10),

    FOREIGN KEY (`user_id`) REFERENCES `users`(`user_id`) ON DELETE CASCADE,

    INDEX `idx_questions_user_id` (`user_id`),
    INDEX `idx_questions_created_at` (`created_at`),

    -- Full-text search index for exact match search mode
    FULLTEXT KEY `ft_questions_search` (`title`, `content`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- --------------------------------------------------------------------------------
-- 3. Question Vectors Table
-- Stores embeddings for the AI Semantic Search feature (Gemini default model).
-- --------------------------------------------------------------------------------
DROP TABLE IF EXISTS `question_vectors`;
CREATE TABLE `question_vectors` (
    `vector_id` BIGINT AUTO_INCREMENT PRIMARY KEY,
    `question_id` INT NOT NULL,
    `source_text` TEXT NOT NULL, -- Text used to generate the embedding
    `embedding` JSON NOT NULL,   -- Gemini embedding vector
    `status` VARCHAR(20) DEFAULT 'ready', -- e.g., 'ready', 'pending', 'failed'
    `created_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    `updated_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    FOREIGN KEY (`question_id`) REFERENCES `questions`(`question_id`) ON DELETE CASCADE,
    UNIQUE KEY `uniq_question_vectors_question_id` (`question_id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- --------------------------------------------------------------------------------
-- 4. Answers Table
-- Stores answers to questions.
-- --------------------------------------------------------------------------------
DROP TABLE IF EXISTS `answers`;
CREATE TABLE `answers` (
    `answer_id` INT AUTO_INCREMENT PRIMARY KEY,
    `question_id` INT NOT NULL,
    `user_id` INT NOT NULL,
    `content` TEXT NOT NULL, -- Content including code sections
    `created_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    `updated_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,

    FOREIGN KEY (`question_id`) REFERENCES `questions`(`question_id`) ON DELETE CASCADE,
    FOREIGN KEY (`user_id`) REFERENCES `users`(`user_id`) ON DELETE CASCADE,

    INDEX `idx_answers_question_id` (`question_id`),
    INDEX `idx_answers_user_id` (`user_id`),
    INDEX `idx_answers_created_at` (`created_at`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- --------------------------------------------------------------------------------
-- 5. RAG: user-owned PDF documents, text chunks, and chunk embeddings
-- --------------------------------------------------------------------------------
DROP TABLE IF EXISTS `documents`;
```

```sql
CREATE TABLE `documents` (
    `document_id` INT AUTO_INCREMENT PRIMARY KEY,
    `user_id` INT NOT NULL,
    `title` VARCHAR(512) NOT NULL,
    `mime_type` VARCHAR(128) NOT NULL DEFAULT 'application/pdf',
    `storage_path` VARCHAR(1024) NOT NULL,
    `byte_size` BIGINT NOT NULL DEFAULT 0,
    `status` VARCHAR(20) NOT NULL DEFAULT 'pending',
    `error_message` TEXT NULL,
    `created_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    `updated_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    FOREIGN KEY (`user_id`) REFERENCES `users`(`user_id`) ON DELETE CASCADE,
    INDEX `idx_documents_user_created` (`user_id`, `created_at`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

DROP TABLE IF EXISTS `document_chunks`;
CREATE TABLE `document_chunks` (
    `chunk_id` BIGINT AUTO_INCREMENT PRIMARY KEY,
    `document_id` INT NOT NULL,
    `chunk_index` INT NOT NULL,
    `content` TEXT NOT NULL,
    `page_start` INT NULL,
    `page_end` INT NULL,
    `created_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (`document_id`) REFERENCES `documents`(`document_id`) ON DELETE CASCADE,
    UNIQUE KEY `uniq_document_chunks_doc_index` (`document_id`, `chunk_index`),
    INDEX `idx_document_chunks_document_id` (`document_id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

DROP TABLE IF EXISTS `document_chunk_vectors`;
CREATE TABLE `document_chunk_vectors` (
    `chunk_vector_id` BIGINT AUTO_INCREMENT PRIMARY KEY,
    `chunk_id` BIGINT NOT NULL,
    `source_text` TEXT NOT NULL,
    `embedding` JSON NOT NULL,
    `status` VARCHAR(20) NOT NULL DEFAULT 'ready',
    `created_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    `updated_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    FOREIGN KEY (`chunk_id`) REFERENCES `document_chunks`(`chunk_id`) ON DELETE CASCADE,
    UNIQUE KEY `uniq_chunk_vectors_chunk_id` (`chunk_id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

SET FOREIGN_KEY_CHECKS = 1;
```